Master's thesis living draft

E-graphs modulo theories

with applications to AC

Sofia Brookie
sofia.ma@protonmail.com

Supervisor: Nicholas Smallbone
Examiner: David Sands

April 21, 2026

Contents

Chapter 1

Overview

1.1 Introduction

This is the 'planning', aka 'living draft' version of this document. This contains notes, planning, and halftime report content besides the eventual thesis content.

E-graphs are a data structure for equational theorem proving, and which provide the basis for *equality saturation* (section 2.3), a method to automatically derive equational consequences from a starting set of terms, equations, and identities. E-graphs have generated a lot of interest since the publication of the Egg library developed by Willsey et al. and their corresponding paper ([6]).

However, E-graphs struggle with a combinatorial explosion when reasoning about certain theories. A notorious case is that of AC theories, those containing associative and commutative operators: the space complexity of saturation is doubly-exponential in the size of the input in the worst case, as shown in section 1.3.

Can equality saturation for AC theories be done in a more efficient way? We build on the proposal for *E-graphs Modulo Theories* (EMTs) from Zucker [7], who suggests integrating a *theory-specific* reasoning method for the AC part with a general E-graph in order to efficiently deal with AC theories. The hope is that specific methods for AC theories can avoid the inefficiencies in using the general equality saturation mechanism for the AC identities.

EMTs are inspired by the success of theory-specific methods in other automated reasoning tasks, such as in *Satisfiability Modulo Theories* solvers (SMT solvers), and the merits of a theory-specific approach in automated reasoning are advocated for in Shankar [3]

We will explore this issue and show that it *is* possible to reason with AC theories and a variety of closely related theories in a way that avoids the worst inefficiencies of plain E-graphs.

In section 4.3, we also provide a formal definition of EMTs that we hope can provide the basis for future work on integrating other theories in a similar manner.

We then, in section 7, discuss an implementation of EMTs and benchmark it against pure E-graph reasoning on a variety of equational reasoning tasks.

Central to our notion of a 'good' theory for an EMT is the necessity of a solver for the word problem over the theory, which we justify in section 4.2. This gives us a lower bound on EMTs for AC and polynomial: the word problem in these cases requires in the worst case exponential space and doubly-exponential time. This is an improvement over the doubly-exponential space upper bound for E-graphs, but highlights that concrete efficiency gains derive from the potential for typical problems to be solvable efficiently. Our solver for AC and polynomials is Buchberger's algorithm, which has received significant effort to achieve good common-case performance.

On the other hand, for the theory ACUN (AC with unit and negation, aka the theory of Abelian groups) or the theory of vector spaces, we have polynomial time bounds: we can use algorithms for Hermite normal form (for Abelian groups) and Gaussian elimination (for vector spaces).

FiXme: Evaluate intro once the thesis is more final. FiXme: Introduction now merged with outline. Forward refs need to be added/deleted.

1.2 An introduction to E-graphs and equality saturation

1.2.1 Equational theorem proving

E-graphs are a data structure to handle *equational theorem proving*: we are given a set of *terms*, a set of *equations*, and a set of *identities*, and we try to determine if some equation follows from the conjunction of these equations and identities.

An equation between terms s and t will be written as $s = t$. An **identity** is a universally quantified equation: for instance, $\forall xy.x + y = y + x$ is an identity, asserting that the equation $x + y = y + x$ holds universally in x and y. To be more precise, an identity represents the set of all of its *ground instances*, which are equations derived from substituting terms for the variables. For instance, if s and t are terms, $s + t = t + s$ is an instance of the previous identity.

We will be concerned with a set of terms generated by some *constants*, $a, b, c, \ldots$, and some *function symbols*, $f, g, h, \ldots$ such that each function symbol has an arity (number of arguments) and, for instance, we can construct the term $f(s, t)$ if s and t are terms and f has arity 2. For a more formal definition, see section 2.5.

A typical equational theorem proving problem looks like $I, E \vdash s = t$, where I is a set of identities and E is a set of equations, and we need a procedure to determine if the equation $s = t$ follows (indicated by the symbol $\vdash$) from I and E.

Equations are easy, in the sense that the congruence-closure algorithm 1.2.2 is already enough to solve the problem entirely when we have only equations (and so $I = \{\}$).

Identities are what makes things tricky: given an identity $\forall x_1 \ldots x_n . s = t$, we need to decide which substitutions for $x_1 \ldots x_n$ we want to make in order to apply the identity. Given that there are infinitely many substitutions, deciding which ones to use is difficult, and in general, equational theorem proving with identities is undecidable (FiXme: citation needed?).

Still, using equality saturation (2.3), E-graphs provide a framework to tackle the problem: depending on the exact identities used, E-graphs can be used as a decision procedure, a semi-decision procedure, or an incomplete procedure for exploring the space of equalities.

E-graphs have a convenient feature: we don't need to provide the goal $s = t$ up-front! E-graphs will in general take the identities, equations, and possibly some terms of interest, and attempt to find all equations between relevant terms that hold. If equality saturation terminates, all possible equalities between the relevant terms has been discovered.

FiXme: The prior 3 paragraphs are not great

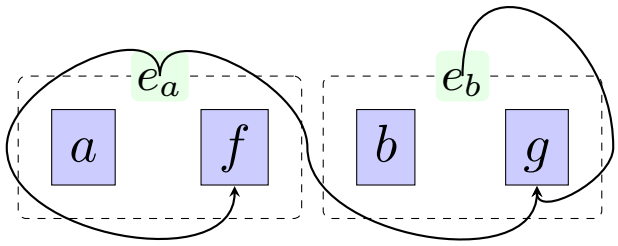
1.2.2 E-graphs for congruence-closure

Consider the input set of equations

$$\begin{aligned} f(a) &= a, \\ g(f(f(a)), b) &= g(a, g(a, b)) \\ g(f(a), b) &= b \end{aligned}$$

The E-graph representing this set of equations is as follows:

Each dashed box is called an **E-class** contains multiple equivalent **E-nodes**.



Each E-node doesn't represent just a term, but a set of terms: for instance, the E-class e_a contains the E-node $f(e_a)$, which represents the set of terms $\{f(t) \mid t \in e_a\}$. Indeed, this is an infinite set, containing $f(a), f(f(a)), f(f(f(a)))$, and so on.

The algorithm to build the E-graph to represent a set of input equations is known as **congruence-closure** and will be described in detail in section 2.2. Note that by 'following the arrows', the term $g(a, g(f(a), g(f(a), b)))$ is represented by the class e_b: this is not part of the input equations, but a consequence of them that can be read off from the E-graph once the congruence-closure algorithm has been applied.

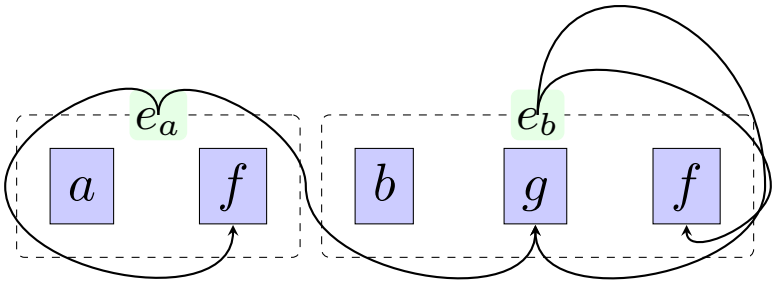
1.2.3 E-graphs for equality saturation

Suppose instead of the equations $f(a) = a$ in the above example we had the identity $\forall x.\ f(x) = x$. To do equality saturation with this identity, we would *match* the left-hand-side $f(x)$ against our E-graph, which would find the matches $f(f(a))$ and $f(a)$, corresponding to the substitutions $x = f(a)$ and $x = a$ respectively. Then, we apply the same substitution to the right hand side, thus finding the *equations* $f(f(a)) = f(a)$ and $f(a) = a$. Inserting these equations into our E-graph and using congruence closure would then yield the above E-graph. Because equalities are symmetric, we would also have to find every match t for x and add the equation $f(t) = t$.

Equality saturation would terminate with the following graph, as for every match in the E-graph of either side of $\forall x.\ f(x) = x$, the other side is in the graph and in the same E-class.

1.3 E-graphs and the problems of AC

FiXme: Rewritten, but the diagrams need redoing FiXme: Too wordy? I could just remove all the exposition in the first three diagrams... While equality saturation on E-graphs can work with any set of identities, operators that are



associative and commutative lead to a worst-case doubly-exponential explosion in the size of the E-graph when it is saturated. The **associativity** identity for an operator f is $\forall xyz.\ f(f(x,y),z) = f(x,f(y,z))$, and the **commutativity** identity is $\forall xy.\ f(x,y) = f(y,x)$.

Even if we do not run to saturation, it is likely that a large number of 'useless' associativity/commutativity steps will have been taken.

We will show some examples of E-graphs handling an AC function symbol $+$.

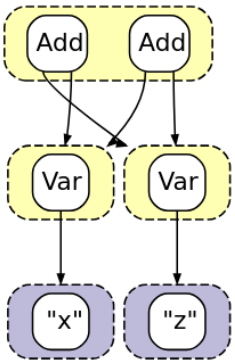


Figure 1: E-graph of the term $a + c$ after rewriting to saturation.

This graph shows that under AC we typically have both a lot of E-classes and a lot of E-nodes per class.

To give an example of a theory with both AC and 'interesting' equalities, we extend the theory from above (containing only the function Add and the AC equations) with a function f, satisfying $\forall xy.f(x+y)+1 = f(x)+f(y)$. We

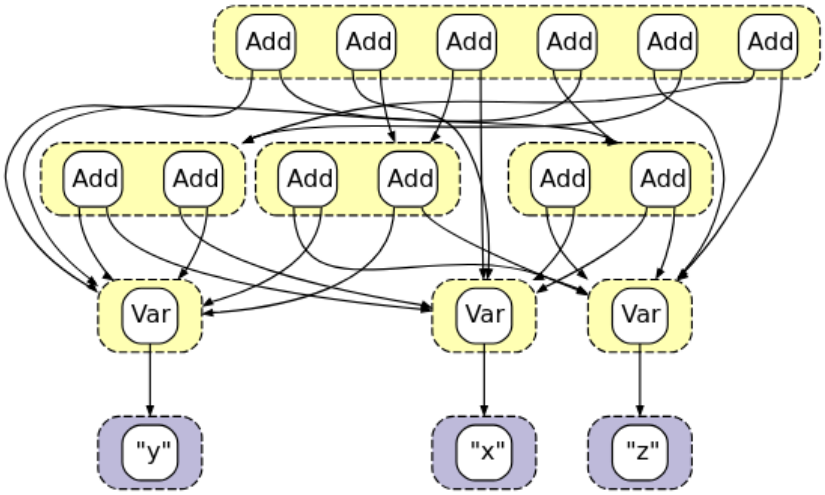


Figure 2: E-graph of the term $a + (b + c)$ after rewriting to saturation.

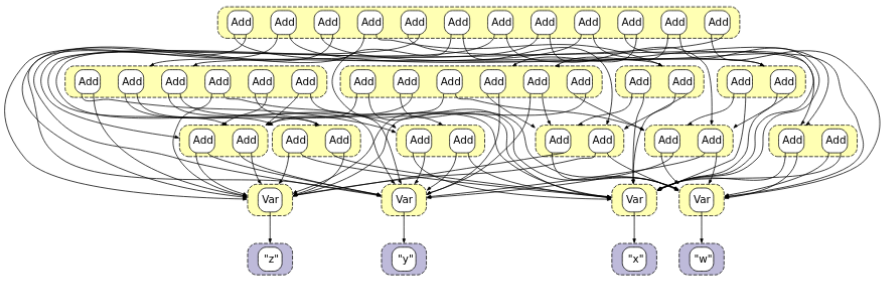


Figure 3: E-graph of the term $a + ((b + c) + d)$ after a number of rewriting steps.

also allow integer constants, and allow the E-graph to reason about integer arithmetic, e.g. that $1 + 1 = 2$. We begin with the term $f(a) + (f(b) + f(c))$. We would like to discover that this input term is equal to $2 + f(a + (b + c))$, using the reasoning

$$
\begin{aligned}
&f(a) + (f(b) + f(c)) = \\
&f(a) + (f(b + c) + 1) = \\
&(f(a) + f(b + c)) + 1 = \\
&(f(a + (b + c)) + 1) + 1 = \\
&f(a + (b + c)) + (1 + 1) = \\
&f(a + (b + c)) + 2 = \\
&2 + f(a + (b + c))
\end{aligned}
$$

As we can see in Figure 4, the E-graph is on the way to discovering this, having for instance generated an equivalence class for the constant 2, but hits the demo's limit again before completing this proof. While this example may be a toy example on a rather limited demo, it shows the extent to which A/C can clutter up the E-graph with many nodes that are not essential to a desired proof. Our equational proof above only needs to use associativity twice and commutativity once.

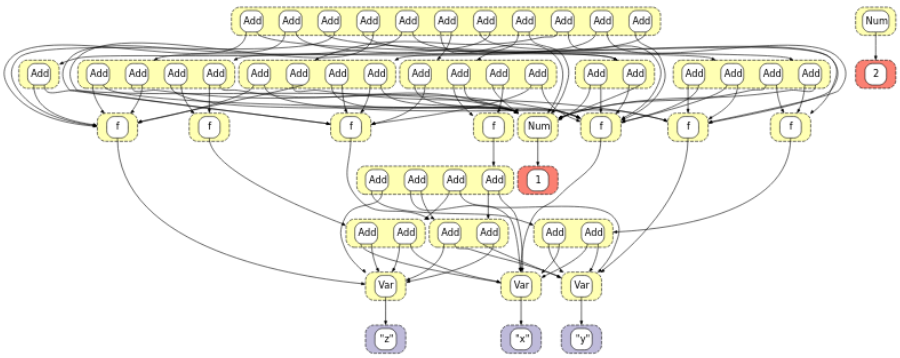


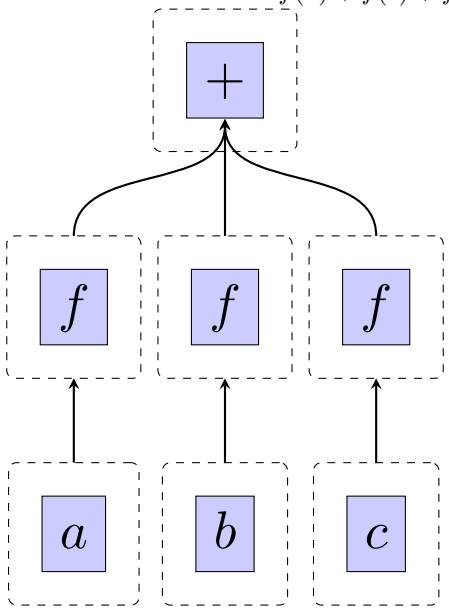
Figure 4: E-graph of the term $f(a) + (f(b) + f(c))$ after a number of rewriting steps.

1.4 Handling AC with EMTs

We will present a pictorial overview of how we might solve the above problem using EMTs rather than E-graphs.

Figure 5 begins with the term $f(a) + f(b) + f(c)$. Since we are implictly working with $+$ as an AC function symbol, we don't represent a particular association of it in the E-graph: we just treat the initial term as the addition of three nodes. We also omit the labels of equivalence classes.

Figure 5: EMT of the term $f(a) + f(b) + f(c)$.



Now, in order to *match* the left-hand side of $f(x) + f(y) = f(x + y) + 1$, we need to do *matching modulo theories*, which will tell us of six matches: $(x, y) \in \{(a, b), (a, c), (b, c), (b, a), (c, a), (c, b)\}$. Then we will add the six substituted forms of $f(x + y) = 1$. Here, we will want to *insert modulo theories*, in a way that keeps the E-graph from representing multiple AC-equivalent terms. The arrows in Figure 6 have become crowded, but top E-class now represents the original term, and $f(a+b)+f(c)+1$ and its variants with a, b, c permuted.

Now, we once again try to match $f(x) + f(y)$. The only new matches will be $(x, y) = (a + b, c)$ and its permutations, and, modulo AC, the substituted-for term $f(x + y) + 1$ will be the same for all of these. So, we just need to add a single node.

Our last step is to notice that the term $f(a + b + c) + 1 + 1$ has, up to AC, the implicit subterm $1 + 1$, which we can reduce to 2. Having reduced the implicit subterm, we no longer need the 3-part term $f(a+b+c)+1+1$, and can entirely replace it with $f(a + b + c) + 2$.

The final EMT (Figure 7) is saturated, and has 'discovered' our desired equality.

Figure 6: EMT of the term $f(a) + f(b) + f(c)$ after some steps.

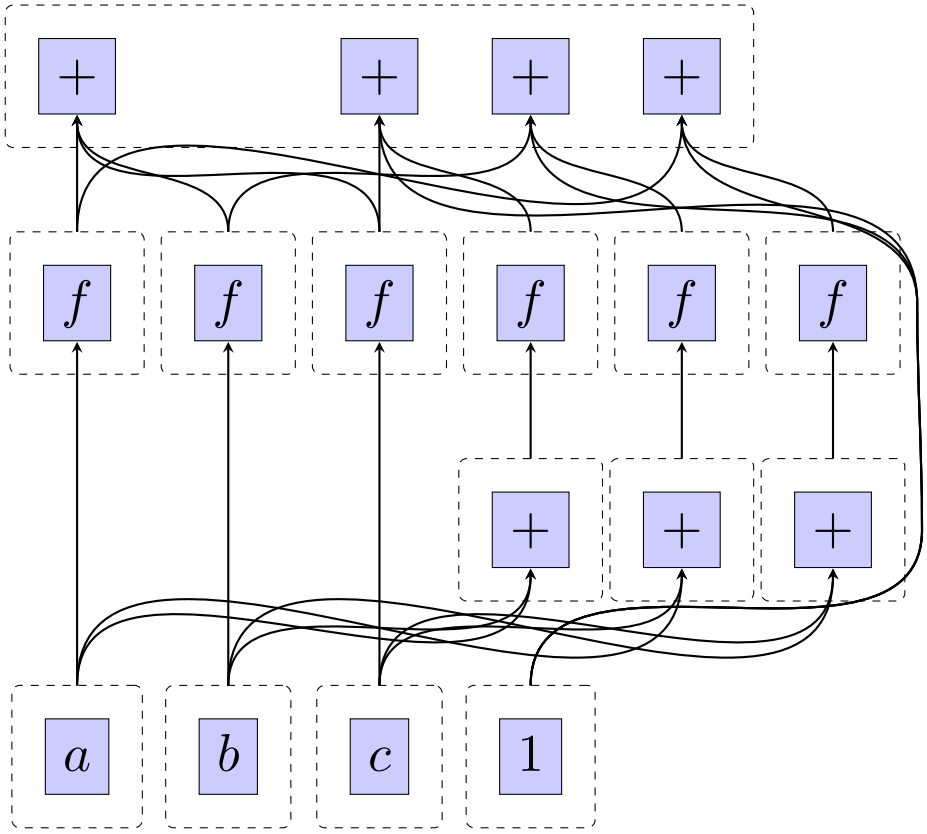
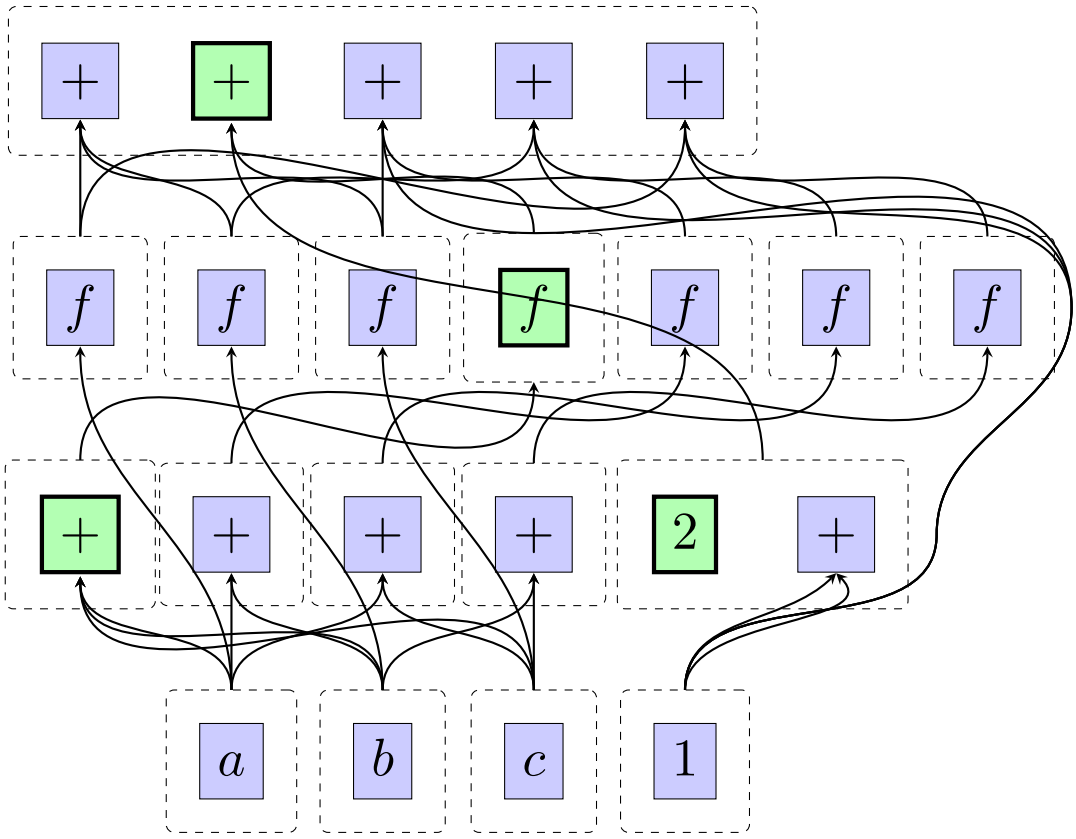


Figure 7: EMT of the term $f(a) + f(b) + f(c)$ once saturated, highlighting the desired term.



Chapter 2

E-graphs in depth

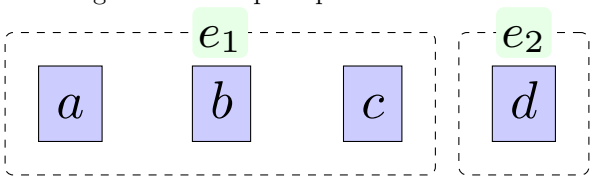
FiXme: This needs a pass to improve the quality. For now, it is something of a dumping ground of 'elementary' material

2.1 Equivalence relations and Union-find

The handling of E-classes, which are equivalence relations of E-nodes, is a central part of the E-graph data structure.

Figure 1 shows a union-find structure that assigns the name e_1 to the equivalence class that encompasses the terms a, b, and c, and the name e_2 to the equivalence class containing just the term d.

Figure 1: A simple equivalence relation



If we abstract from the contents of the E-nodes, we can just think of each E-node as a unique *constant*. Then the problem is to determine if an equation between constants $a = b$ follows from a set of equations $\Gamma = \{c = d, \ldots e = f\}$. For this problem, we have a beautiful solution that comes with an efficient implementation. This is what we would love to see for every fragment, but things become more difficult as we introduce more complexity.

We recall some standard notions. Given a set, an **equivalence relation** over

that set is a reflexive, symmetric, and transitive relation. Given an equivalence relation, we can take the **quotient** of the set by it, a set containing a single element for each **equivalence class** of elements in the original set.

The *Union-Find* data structure is a very efficient means to represent an equivalence relation over a set, and to update it either by extending the underlying set or by merging two equivalence classes. Union-find avoids any fiddly manipulation with reflexivity, symmetry, and transitivity, and focuses on a *semantic* characterization of equivalence relations: a function $r : A \to B$ splits the domain A into equivalence classes based on the equality of their images in B: define R by $R = \{(a, b) \mid r(a) = r(b)\}$. In this, B *is* just the set of equivalence classes, and r *is* just the quotient.

Using this fact, we can sketch an implementation that maintains this mapping r from a set of constants to a set of equivalence classes. To start off with, we don't need to represent the codomain B in any fancy way: we just assign a new identifier to each equivalence class. Then, a basic implementation is just an array or table with indexes in A and entries in some set of identifiers for equivalence classes.

However, this representation potentially requires a bit of work to update: if we want to take into account a new equation $a = b$ with an existing r, we need to set all things mapping to $r(a)$ and $r(b)$ to the same identifier. The solution is to not store r itself, but a function q such that the fixed point q^* equals r. To spell this out, $r(q(a)) = r(a)$ for all $a \in A$, and since A is finite, there will always be some number n such that $r(a) = q^n(a) = q(\ldots(q(a)))$.

The trick is to then use *path compression*: whenever we look up $r(a) = q(\ldots(q(a)))$, we alter some of $a, q(a), q^2(a), \ldots$ to be directly mapped to $r(a)$. This is the essence of the **Union-Find** data structure for representing equivalence classes, and it is an example of a self-optimizing data structure: each access ensures that the number of iterations required to reach a fixed point of q is reduced on-the-fly.

We won't spell out the exact details of known path-compression techniques here. The end result, if the path compression is managed appropriately, is that n operations, if they are either calculating the representative of a term or merging two equivalence classes, takes just $O(n\alpha(n))$, where α is the inverse Ackermann function, a very slow-growing function FiXme: source. There are a few variations on the exact method for path compression, but all union-find data structures use the same elegant core idea.

2.2 Congruence relations and Congruence-closure

A congruence relation on a set of terms is one that respects the function symbols: for instance, if f is a function symbol of arity 3, and $s_1 = t_1$, $s_2 = t_2$, and $s_3 = t_3$,

then $f(s_1, s_2, s_3) = f(t_1, t_2, t_3)$. In general, we get the following axiom schema: for each n-ary symbol f, $a_1 = b_1, \ldots, a_n = b_n \vdash f(a_1, \ldots, a_n) = f(b_1, \ldots, b_n)$. This is the larger fragment solved by a congruence-closure data structure.

Comparing with the case of 0-ary function symbols, we seek a semantic characterization of congruence relations, i.e., congruent equivalence relations. However, no existing approach compares in elegance to union-find.

We keep a union-find storing equivalence classes of **E-nodes**[1]: as we have seen, if $s = t$, then we don't need to waste space represent both $f(s)$ and $f(t)$, given that these are identical. So, we restrict ourselves to keeping not a set of terms, but a set of 'lifted' terms like $f(e_1)$, if e_1 is the name we assign to the class containing s and t.

This has the potential to save an infinite amount of space: if we have the input equation $f(a) = a$, then the equivalence class of a up to congruence includes the infinite set of terms $a, f(a), f(f(a)), \ldots$. So the 'lifting' of terms to act on equivalence classes lets us represent every congruence relation in a finite way.

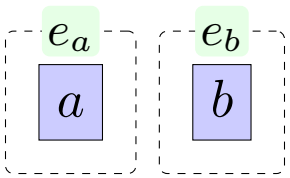
Indeed, this idea yields the congruence-closure algorithm for deciding equality over any set of equations. We will simply refer to the congruence-closure structure as an **E-graph**, although the term E-graph is commonly used when we also include equality saturation (2.3).

We will show the congruence-closure algorithm running on our previous set of equations

$$\begin{aligned} f(a) &= a, \\ g(f(f(a)), b) &= g(a, g(a, b)) \\ g(f(a), b) &= b \end{aligned}$$

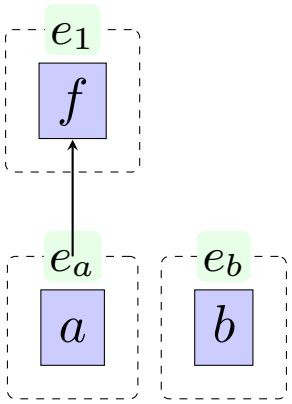
We can build the E-graph over these equations as follows:

First, we add all terms with no subterms, namely a and b. These each get a distinct equivalence class.

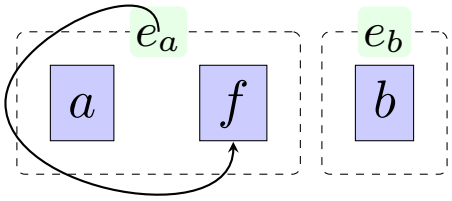


[1] 'E' in E-graph, E-node, E-class, etc., stands for equality

Next we add $f(a)$: a is represented by e_a, so we represent $f(a)$ by $f(e_a)$, and can graphically depict this with an arrow:



Now we have represented both terms in the equation $f(a) = a$, so we merge the resulting equivalence classes:



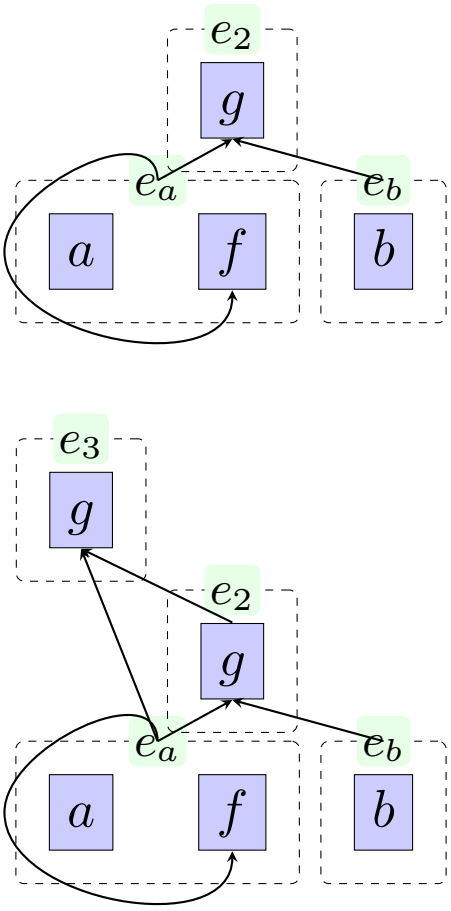
This merged class represents all of $a, f(a), f(f(a)), \ldots$.

Now we look at representing some of the input terms using the function symbol g. Using our existing equivalence classes, $f(f(a))$ is represented by $f(f(e_a)$ is represented by $f(e_a)$ is represented by e_a, so $g(f(f(a)), b)$ is represented by $g(e_a, e_b)$, which is not yet in our E-graph. $g(e_a, e_b)$ represents all of $g(f(f(a)), b)$, $g(a, b)$, and $g(f(a), b)$.

The last term to represent is then $g(a, g(a, b))$, which is represented by $g(e_a, e_2)$.

Now, we need to process the remaining two equations. $g(f(f(a)), b) = g(a, g(a, b))$ tells us to merge the equivalence classes e_2 and e_3. $g(f(a), b) = b$ tells us to merge e_2 and e_b.

The last step is to apply to rule of *congruence*. Our graph now has two nodes representing $g(e_a, e_b)$. We merge them; as they are already in the same class, there is no further work to be done:

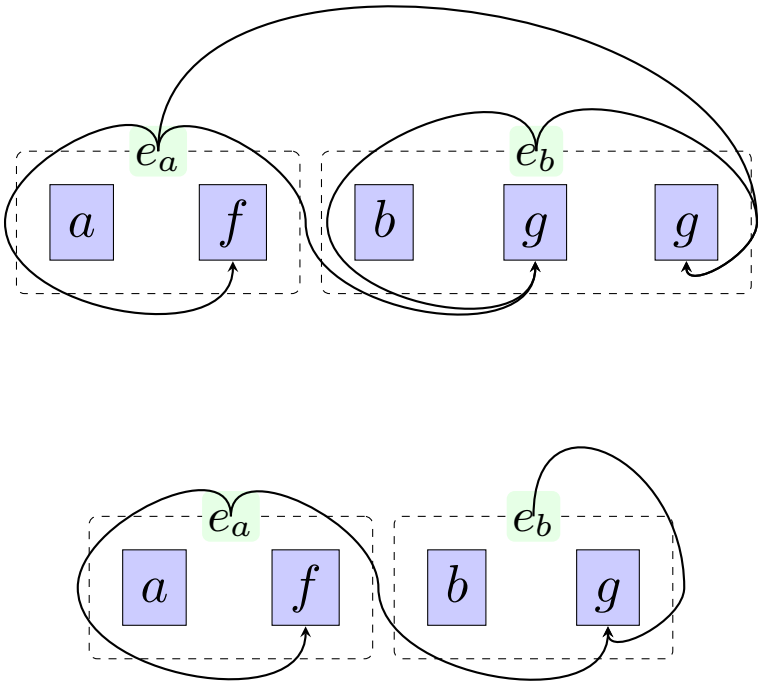


However, in some cases applying congruence can cause a chain of merges. This process terminates as there is a maximal state of finite size (an E-graph assigning all nodes to the same equivalence class), so any potential infinite chain of congruence steps will reach a fixed point after finitely many steps.

Note the final E-graph has two *cycles*: $f(e_a) = e_a$ and $g(e_a, e_b) = e_b$. In general, we might have cycles going through multiple intermediary nodes: $f(\ldots g(\ldots e_1 \ldots) \ldots) = e_1$. Cycles become very important in the analysis of how theories fare with E-graphs under equality saturation (section 3.0.2).

2.2.1 A more formal description

We will maintain a union-find structure representing a quotient function $r : E \to E/R$; E is some superset of the constants A that includes an element for every E-node. We will call these **E-nodes**, and elements of the set E/R **E-classes**.



We introduce, for each n-ary function symbol f, a mapping $f_r : (E/R)^n \to E/R$. Unfortunately, we now have two objects to keep in sync if we hope to represent r via the efficient union-find structure from before. In particular, if r is altered to merge two equivalence classes, we then have to scan through our representation of f_r to see if we encounter a case where two equal tuples are mapped to unequal representatives. In that case, we must merge the representatives according to congruence, and then go back and check if this new merge has any consequences, and so forth.

This system, where we bounce new equalities between data structures, is rather typical of **theory combination** FiXme: add ref to later part.

2.2.2 Congruence closure builds a rewrite system

We can read off a set of equations from our final E-graph:

$$
\begin{aligned}
a &= e_a \\
f(e_a) &= e_a \\
b &= e_b \\
g(e_a, e_b) &= e_b
\end{aligned}
$$

Given this resulting set of equations, we can determine if any two terms at all are equal under our initial set of equations: simply reduce the terms by treating these equations as a *rewrite system*, read from left-to-right, and see if the normal forms are the same. FiXme: Refs for connections to rewrite systems.

2.3 Equality saturation

Now that we can reason about the logical consequences of (ground) equations, we look to solve the problem of reasoning with identities. The method of **equality saturation** is what we will focus on in this thesis.

The key idea is that, given a representation of a congruence relation, we *E-match* each identity $\forall a_1, \ldots, a_n. s_1 = s_2$ against the congruence relation to find all instances $s_1[t_1/a_1, \ldots, t_n/a_n]$ and $s_2[t_1/a_1, \ldots, t_n/a_n]$. Then, we find the representatives of these two terms. If they are not equal, we have found two equivalence classes to merge to take into account the identity. We will refer to this as a **proper merge**.

Unfortunately, proper merges are not sufficient. If we consider the associative identity $\forall abc.\ a + (b + c) = (a + b) + c$, and we begin with the two equalities $u = w + (x + (y + z))$ and $v = ((w + x) + y) + z$, we would like to prove the theorem that

$$
\begin{aligned}
(\forall abc.\ a + (b + c) = (a + b) + c), \\
u = w + (x + (y + z)), \\
v = ((w + x) + y) + z \\
\vdash u = v
\end{aligned}
$$

However, the attempt to prove even the first step, that $w + (x + (y + z)) = (w + x) + (y + z)$, founders. Our representation includes no equivalence class to represent $(w + x) + (y + z)$, and so there are no equivalence classes to merge.

The 'solution' is to introduce the possibility of **inserting** new classes during the procedure. For instance, we can find an instance $s_1 = s_2$ of an identity such

that s_1 maps to a known equivalence class but s_2 doesn't. We seed s_2 as a new term by calculating its equivalence class, potentially introducing a new class if no prior one exists, and then merging the equivalence classes of s_1 and s_2. We call this mix of seeing and a proper merge an **improper merge**.

Insertion has one key problem: we now have no guarantees the process of merging will terminate. With only proper merges, there are a finite set of input terms and thus a finite bound on the equivalence relation we maintain. Thus, eventually we will get to a point where no merge changes the equivalence relation, and we have reached **saturation**. Introducing new classes for inserted terms now means that saturation is no longer guaranteed to be reached at any point, and worse, the strategy for insertion and merging now can affect whether or not saturation is reached.

To even get a *semi-decision procedure*, where we aim to prove a specific equality if it exists and can fail otherwise, we must ensure that the insertion and merging processes are sufficiently *fair* (in the sense of 'fair' as used in concurrent programming): we must have some strategy to ensure every possible term that could be inserted *does* get inserted eventually, or we might get stuck inserting other terms infinitely while nonetheless not making progress on the goal.

In some sense, this is to be expected, as even for just equational theorem proving, there are theories which are known to be undecidable. However, even when the process of E-matching, inserting and merging doesn't terminate, we can consider the infinite fixed-point of this process as the 'semantics' of our E-graph [4], and this will be what we will preserve in the shift to EMTs. This process will be referred to as the 'equality saturation outer loop'. A schematic form is as follows:

1. Insert all the starting terms into the E-graph
2. Repeat until fixed point:
   (a) **E-match** an identity against the E-graph, finding all ground instances of one side of the identity that exist in the E-graph.
   (b) **Insert** Using the substitution from the previous step, construct the term resulting from applying the substitution to the other side of the identity. If it is not already represented by the E-graph, add it to the E-graph.
   (c) **Merge** the E-classes representing both sides of the equality above.
   (d) **Rebuild** the graph, using the rules of congruence closure

If the fixed point is reached, we call the graph **saturated**.

2.3.1 A few variations

We can do several matches/insertions/merges in between rebuilding; it is sufficient that rebuilding is *eventually* run after changes to the E-graph [6].

There are plenty of extensions to the outer loop that include things such as conditional reasoning (along the lines of: if $a = b$, then assert that $c = d$ by merging classes) or E-class analyses ([6]).

2.4 E-graphs as tree automata

FiXme: Things about rational sets, tree automata mod theories? and recognizable vs rational?

[1] [5]

A much better source: [4]. Actually, I should use this source more often...

2.5 Terms, signatures, and theories formalized

FiXme: Where should this go?

A **signature** is a set Σ of function symbols with an arity given by a function $\alpha : \Sigma \to \mathbb{N}$. Given a set of constants A, a **term fragment** $\Sigma(A)$ is a pair of an element of $f \in \Sigma$ together with a list of elements of A of length $\alpha(f)$. A **term** over A, denoted by $\Sigma^*(A)$, is defined inductively as either a constant $a \in A$ or a term fragment over $\Sigma^*(A)$.

Given a term in $\Sigma^*(A)$, we can extend it to be over more constants $\Sigma^*(A + B)$ in the obvious way, denoting by $+$ the disjoint union of sets. We will often use auxiliary constants written as a, b, c etc., but these should be distinguished from the notion of a variable in an identity defined below.

An interpretation of a signature Σ is a *carrier* set A together with a function $a : \Sigma(A) \to A$ that interprets term fragments, from which the interpretation of terms in $\Sigma^*(A)$ is given by recursive application of a, interpreting from the leaves to the root in the natural way. We will denote this by $\mathbf{fold}(a) : \Sigma^*(A) \to A$.

Given a signature, there is a standard notion of first-order logic[2] over the terms of $\Sigma^*(A)$. More specifically, we will consider **theories**, which are conjunctions of identities, where an **identity** is a proposition of the form $\forall x_1.\cdots\forall x_i.t_1 = t_2$ where t_1 and t_2 are terms over $A + \{x_1, \ldots x_i\}$. Note that the number of quantifiers can also be 0. A **ground instance** of an identity is an equation of the form $\sigma(t_1) = \sigma(t_2)$, where σ is a **substitution** that maps variables in $\{x_1, \ldots x_i\}$ to

[2] given the fragment we will consider in this thesis, there is no actual distinction between classical and intuitionistic first-order logic. [source about Horn(CL) = Horn(I)]

terms over A. We will generally use **equation** to mean any equation between terms in the **ground fragment** $\Sigma^*(A)$, while reserving identity for universally quantified equations.

Example. Let $\Sigma = \{f, g\}$ and $\alpha(f) = 2, \alpha(g) = 1$. If $A = \{x, y, z\}$, then $f(x, y)$, $g(z)$, and $f(y, y)$ are elements of $\Sigma(A)$. In addition, x, $f(f(x, y), g(x))$, and $g(g(g(z)$ are elements of $\Sigma^*(A)$.

If we have the theory $T = (\forall ab.f(a, b) = g(f(a, b))) \wedge (\forall a.g(a) = g(g(a)))$, then we can derive the ground equation $f(x, g(y)) = g(f(x, g(y))$ from T by a substitution in the first identity of $\sigma(a) = x, \sigma(b) = g(y)$.

2.6 Matching, merging, seeding, and canonicalization formalized

FiXme: To be moved Here we spell out the relevant procedures in some detail, given as they will be central to our generalization to EMTs 4.3.

A congruence closure structure $(A, E, b : A \to E, q : E \to E, c : (f : \Sigma) \rightharpoonup E^{\alpha(f)} \to E)$ consists of a set of constants A, a set of 'E-node names' E, a name for each constant $b : A \to E$, a union-find map q, and a partial function-symbol map c, where $c_f : E^{\alpha(f)} \to E$ is the function map for a particular symbol $f \in \Sigma$. Particular entries $c_f(e_1 \in E, \ldots, e_n \in E) \mapsto e \in E$ are called **E-nodes**. Fixed points of the map q are called **E-classes**.

We will use the term E-graph both for this abstract structure as well as concrete implementations, for which some auxiliary maps are useful to speed up certain operations.

The relation $=_q$ is defined as $\{(a, b) \mid q^*(a) = q^*(b)\}$ where q^* is the fixed point of q.

A term fragment $f(e_1, \ldots, e_n) : \Sigma(E)$ is represented by the E-class e in an E-graph if $c_f(e_1, \ldots, e_n)$ maps to some f where $f =_q e$. A term $t \in \Sigma^*(A + E)$ is represented by e if it is simply a node name $f \in E$ such that $f =_q e$, or if it is a constant $a \in A$ and $b(a) =_q e$, or if it is a term fragment $f(t_1, \ldots, t_n)$ such that t_i are represented by e_i and $f(e_1, \ldots, e_n)2$ is represented by e.

The matching problem is the following: for a pattern $p \in \Sigma^*(A + S)$, is there a substitution $\sigma : S \to \Sigma^*(A + E)$ such that the term $p\sigma \in \Sigma^*(A + E)$ is represented by the E-graph?

Merging is the process of taking two elements $e_1, e_2 \in E$ and updating q so that $e_1 =_q e_2$.

Seeding takes a term t and updates the E-graph so that there is some $e \in E$

such that e represents t.

Canonicalization updates q and the function-symbol maps c_f to ensure that if $e_1 =_q e_2, \ldots, e_{n1} =_q e_{n2}$, then $c_f(e_1, \ldots, e_{n1}) =_q c_f(e_2, \ldots, e_{n2})$. In general, this is done by a fixed-point method of finding pairs of tuples such that the corresponding elements are equivalent under $=_q$ while c_f maps them to inequivalent elements, and then merging.

Chapter 3

Which theories do E-graphs struggle with?

AC is a good example of a theory which E-graphs handle poorly. However, this is not necessarily the case for every common theory, so it may not always be worth the extra complexity of using EMTs. We list some common identities which are worth analyzing:

Theory name	Abbreviation	(Function symbol, arity)	Theory
Associativity	A	$(f, 2)$	$\forall xyz.f(x, f(y, z)) = f(f(x, y), z)$
Commutativity	C	$(f, 2)$	$\forall xy.f(x, y) = f(y, z)$
Left-unitality		$(f, 2), (e, 0)$	$\forall x.f(e, x) = x$
Right-unitality		$(f, 2), (e, 0)$	$\forall x.f(x, e) = x$
Unitality	U	$(f, 2), (e, 0)$	Both of the above
Left-distributivity		$(f, 2), (g, 2)$	$\forall xyz.g(x, f(y, z)) = f(g(x, y), g(x, z))$
Right-distributivity		$(f, 2), (g, 2)$	$\forall xyz.g(f(y, z), x) = f(g(y, x), g(z, x))$
Distributivity	D	$(f, 2), (g, 2)$	Both of the above
Idempotence	I	$(f, 2)$	$\forall x.f(x, x) = x$
Inverse	N	$(f, 2), (i, 1), (e, 0)$	Unitality and $\forall x.f(i(x), x) = e$ $\forall x.f(x, i(x)) = e$
Involution	V	$(i, 1)$	$\forall x.i(i(x)) = x$

Table 3.1: Definitions of some common identities

We use the convention of naming combined theories by taking the juxtaposition of the theory names: AC for a function symbol f is the theory with both A and C identities, and so forth.

FiXme: cut down if some of these don't appear FiXme: also we don't need the

one-sided versions, probably

This list of identities covers a wide range of common mathematical structures, such as semigroups, monoids, groups, semirings, rings, modules, semilattices, lattices, Boolean algebras, as well as some less common ones such as bands, quasigroups, loops, and near-semirings.

Note that the condition $x \neq 0$ for x to have an inverse in a field is not formalizable in purely algebraic terms, but this list covers every other field identity. A similar observation applies to vector spaces.

3.0.1 Combinatorics

FiXme: This needs a MASSIVE rewrite

3.0.2 How common theories fare with E-graphs

In general, using theories with equality saturation will lead to a growth in the size of the graph, as more nodes will need to be seeded to ensure that all consequences of the identities are discovered. We will analyze typical theories and how they fare.

The possibility of cyclic graphs is a core sticking point for how certain theories fare when used in the equality saturation process.

Minimally-expansive theories

Given a fixed set E of E-classes, the maximum number of E-nodes in a *canonical* E-graph is $|\Sigma(E)|$. We will focus on the sizes of canonical E-graphs. For the theories C, I, and V, every subterm of one side of the identity is also a subterm of the other, and so no new E-classes are ever added as a result of these theories; all merges are proper.

For U and its one-sided subtheories, it is possible the subterm e is not part of the graph and needs to be seeded, but once seeded, the class of e will remain in the graph ensuring that all subsequent merges are proper.

We will call C, I, U, left-unitality and right-unitality **minimally expansive**. A formal statement of this property is that for every equation, each subterm of one side is a subterm of the other or a 0-ary function. Any such theory can increase the number of E-classes by at most a constant amount, and expand the number of E-nodes between classes to at most the maximum $|\Sigma(E)|$. In practice, we can get tighter bounds by simply looking at what E-classes/E-nodes can result; see 3.0.2.

The prime examples of expansivity that we will cover in this thesis are, then, associativity and distributivity.

FiXme: Important find: how does this compare to the notion of 'Weak term acyclicity' from [4]?

Inverse

Inverse is something of an interesting case. Given the identity $\forall x.f(i(x), x) = e$, x does not appear at all on the right-hand side. Given that, an E-graph representing e could be extended to contain *any* new term for x as a result of this identity. However, it would seem unproductive to instantiate x with random terms, and indeed we can reason as follows:

Consider all the proofs that could involve the use of the identity for inverse in this direction. We show that $f(i(s), s) = e$ is only useful for certain values of s:

1. We could use transitivity via $t = f(i(s), s)$ and $f(i(s), s) = e$. In that case, we require already to establish an equation containing $f(i(s), s)$.

2. We could use transitivity via $f(i(s), s) = e$ and $e = t$, proving that $f(i(s), s) = t$. The usefulness of this equation can be analyzed by following this argument recursively.

3. We could use symmetry to establish $e = f(i(s), s)$, but this is again only useful in conjunction with another use of this reversed equality

4. We could use congruence to establish that, if $i(s) = t_1$ and $s = t_2$, then $f(i(s), s) = f(t_1, t_2)$.

A similar argument holds for the symmetric identity $\forall x.f(x, i(x)) = e$ in the theory.

We can conclude that we can achieve completeness if we seed $f(i(s), s)$ and merge it with e only if $i(s)$ and s are already represented by the E-graph. This doesn't add any new nodes of the form $i(s)$ for any s, and so this process terminates.

This form of reasoning about limiting the scope of seeding is rather ad-hoc. Worse, it is not stable under theory combination, as an equation $s = t$ can be an instance of another identity without either s or t being represented by the E-graph, so we must analyze potential 'chains' of reasoning in the combined theory.

Our approach to EMTs is a more principled approach that can handle the theory of inverses and similar theories. FiXme: Remove this if this doesn't end up being true

Summary of simpler theories

- C: E-classes containing n nodes of the form $f(a_1, a_2)$ must expand to include up to n new nodes $f(a_2, a_1)$.
- U: Up to 1 new E-class for e. Every E-class a must expand to include up to 2 new E-nodes, $f(e, a)$ and $f(a, e)$.
- Left- and right-unitality: The same as above, but only at most one new E-node per E-class.
- I: Every E-class a must expand to include up to 1 new E-node, $f(a, a)$.
- V: Up to 1 new E-class for e. For every E-class a such that $i(a)$ is represented, at most one new E-class must be added with the two E-nodes $f(i(a), a)$ and $f(a, i(a))$

FiXme: Other non-expansive theories: CU, CUI, CUV? Can we give a formula for non-expansive combinations?

A and AU

It is a basic mathematical observation that any term under a function symbol f in the theory A can be rewritten into either a fully left-associated or right-associated term just by treating the A identity as a rewrite rule in one direction. For instance, if we use $+$ as our associative binary operator, $(a + (b + c)) + d \mapsto a + ((b + c) + d) \mapsto a + (b + (c + d))$ if we right-associate everything.

Therefore, we can think of terms up to associativity as a non-empty string of their non-$+$ leaf nodes. The number of binary trees that yield a string of length $n + 1$ is given by the n-th Catalan number, which by itself is exponential in n. However, due to the sharing of subterms, things are hardly as bad as this. In general, each of the $O(n^2)$ substrings of the input string will be given a single E-class, each containing $O(n)$ E-nodes for the various ways to divide that substring into two parts.

If we allow cyclic terms modulo A then the E-graph represents an infinite list of strings, of which there are infinitely many distinct E-classes. The problem can be illustrated by an E-graph representing $a + b = a$, for which every string of leaves $a, ab, \ldots, ab^n$ is part of this E-class. This means that the infinite set of strings $b, \ldots, b^n$ form the leaves of distinct E-classes that a standard equality saturation loop will seed.

The most common case this occurs is with the theory AU, where for every term t we have the identity $t = t + 0$ (using 0 as the name for the identity of $+$). In this case, any finite set of seeded terms such as $0, 0 + 0, (0 + 0) + 0$ will, under a fair strategy, eventually be merged into a single E-class by applying the identity 'backwards'. There is unfortunately no direct bound on the intermediate sizes

of the E-graph during this process, and it is dependent on the exact sequence of saturation steps taken. There is also the possibility of a cyclic equation of the form $a + b = a$ being created *without* the E-graph having discovered that $b = 0$, for which even an eager strategy of trying to 'collapse any 0 we see' is not bounded in the intermediate size.

Cancellation

An possible workaround is to add the reasoning principle $\forall xy.x+y = x \Rightarrow y = 0$, which is not purely equational but is not too hard to introduce in an ad-hoc way to our saturation outer loop. Such an identity holds in the theories with inverses or cancellation. Unforunately, this is not sufficient: in general, we would need a rule that says "if x is by some cyclic path in the E-graph equal to $a_1 \ldots + x + \ldots b_n$, then $a_1 + \ldots + b_n = 0$". (The reason for the form with $\ldots$ is that we hope to apply this early, *without* relying on some number of associativity steps to write this into the semantically equivalent form $a+x+b$). However, the introduced equation $a_1 + \ldots + a_n$ is not enough to reduce the infinite number of subterms modulo associativity: for instance, in the case $a + x + b = x$, we have $a, a + a, a + a + a, \ldots$ as subterms with only $a + b = 0$ as an equation to reduce them. FiXme: formally check, please. Also motivate better

Commutativity

If we also require (perhaps a limited form of) commutativity, then the infinite number of equivalence classes collapses, as then the infinite set of terms $x, a+x+b, a+a+x+b+b, \ldots$ can be captured by the derived equations $x+0 = x, a+b = 0$.

However, if we go so far, it is much nicer to simply use the elegant form of E-graphs mod AC developed in FiXme: forward-ref.

Unfortunately, E-graphs mod A are not a panacea here due to decidability issues, but can improve things significantly. FiXme: Expand and forward-ref once more is written

AC and ACU

Given an AC operator $+$ and a starting term AC-equivalent to $a + b + c + \ldots$, then the equivalence class of the starting term will be rewritten to contain the operator $+$ applied to *all partitions into nonempty sets* of $\{a, b, c, \ldots\}$: all non-empty sets A and B such that $A \cup B = \{a, b, c, \ldots\}$ will be given distinct E-classes to represent their sum, and it is clear this equation can be solved for a nonempty B whenever A is neither $\{\}$ nor $\{a, b, c, \ldots\}$ itself.

So, if $|\{a, b, c, \ldots\}| = n$, then to represent AC-equivalence, we will need to seed all $2^n - 1$ E-classes representing all non-empty subsets. And each of these E-classes to represent the subset of size m will have $2^m - 2$ E-nodes, to represent all partitions except for those where either side of the partition is empty.

Summing up, we have $\Sigma_{m=1}^{2^n}(2^m - 2)$ E-nodes, which can be reduced using the geometric series formula to $2^{2^n} - 2 - 2(2^n) = 2^{2^n} - 2^{n+1} - 2$ E-nodes across $2^n - 1$ E-classes.

For ACU, we get a similar situation, just dropping the use of 'nonempty' above. So we have 2^n E-classes and $\Sigma_{m=0}^{2^n}(2^m) = 2^{2^m}$ E-nodes.

FiXme: Polynomials, D, etc

3.1 Orphaned sections

3.1.1 A couple of bad ways to handle AC

FiXme: In general, we need a good section on 'hacks' to handle AC and other theories within a pure E-graph framework

FiXme: This should be moved The Egg library handles the application of identities with a backoff method, where equations to apply are selected based on their weight, and the weight of an equation decreases when it fires. Any equation that suffers from blowup, like associativity or commutativity, will be fired less and less, in the hopes of finding more 'interesting' equalities without having to run to saturation. This method isn't able to properly distinguish between 'interesting' and 'uninteresting' cases of associativity and commutativity, and it's also of no use when running to saturation.

FiXme: This is an old example that refers to our example in the section about AC. It motivated matching modulo theories One could attempt to deal with AC ahead-of-time by automatically generating derived equations like $f(a+b)+(1+c) = (f(a)+f(b))+c$ and $1+f(a+b) = f(a)+f(b)$ from the source equation. This method is not in general complete: in the AC case, we would need infinitely many equations to handle matching at an arbitrary distance, as we can see with the term $f(x+y)+(z_1+(z_2+\ldots(z_n+1)\ldots))$. Here, any pattern that can determine that both $f(x+y)$ and 1 are subterms, and therefore that there is a match modulo AC, must have n auxiliary variables to capture (and ignore) each z_n that stand between the $f(x+y)$ and the 1. This gives some idea of the difficulty in simply preprocessing the input theory and set of terms, and motivates considering a modification to the E-graph representation and algorithms themselves.

3.1.2 Miscellaneous tables

Short name	Common name	Module structure	Algorithm	Notes
AC	Commutative semigroup	$\mathbb{N}^+$-"module"	AC-completion	
ACU	Commutative monoid	$\mathbb{N}$-module	ACU-completion	
ACUN	Abelian group	$\mathbb{Z}$-module	Hermite normal form calculation	
ACUD				

E-graphs	Rewriting	Tree-automata
E-graph	Ground rewriting system	Nondeterministic bottom-up tree automata
Canonical E-graph	Confluent ground rewriting system	Deterministic bottom-up tree automata
Canonicalization by congruence-closure	Ground Knuth-Bendix completion	*speculative:* $\wedge$-determinization
Equality saturation with identities	Split Knuth-Bendix completion	?
speculative: Non-Ground Congruence-Closure Congruence-Closure with Free Variables Normaliztion via Rewrite Closure	Knuth-Bendix completion	*speculative:* RTN-tree-automata
this work: E-graph modulo theories	Rewriting modulo theories	*speculative?:* Tree-automata modulo theories

3.1.3 Miscellaneous notes

Semantic foundations gives the citation "Charles Gregory Nelson. Techniques for Program Verification. PhD thesis, Stanford University, Stanford, CA, USA, 1980. AAI8011683." for E-graphs. FiXme: Check this

Egglog evaluation is seminaïve, so it is incremental. Egg is *not* incremental at all! Even if the de Moura paper is. Also I should add de Moura to the citation database.

Everyone cites Kozen 77 for the NP-completeness of E-matching, but I should make a picture of the actual SAT embedding.

Chapter 4

E-graphs modulo theories

FiXme: Nelson-Oppen and Shostak. Does our approach require stable-infiniteness etc?. Discussion of completeness in theory combination? Equality propagation?

To deal with problematic theories such as AC, we will define the notion of *E-graph modulo theory*. As we have seen FiXme: check that we have, E-graphs deal with AC by explicitly generating every term that is AC-equivalent to some input term, which is always a finite but worst-case doubly-exponential-sized set.

FiXme: Undefined notions incoming

To build up to the notion of EMTs, we consider *critical pairs*, a notion from term rewriting. If we have a rewriting relation $\rightarrow_{\mathcal{R}}$, then a **peak** is a trio of terms s_1, s_2 and t such that $s_1 \leftarrow_{\mathcal{R}} t \rightarrow_{\mathcal{R}} s_2$. A **critical pair** is a peak that cannot be obtained as a non-trivial substitution instance of another.

For instance, if we have the rules $f(x, y) \rightarrow_{\mathcal{R}} g(y, y, x)$ and $h(x) \rightarrow_{\mathcal{R}} x$, then the term $f(h(g(x, y, z)), w)$ is a peak, because

$$g(w, w, h(g(x, y, z))) \leftarrow_{\mathcal{R}} f(h(g(x, y, z)), w) \rightarrow_{\mathcal{R}} f(g(x, y, z), w)$$

Now, E-graphs take a set of equations E and build a rewrite system $\rightarrow_E$ such that $s =_E t$ iff there is a c such that $s \rightarrow_E^* c \leftarrow_E^* t$. If we have a rewriting system with a peak $s_1 \leftarrow_{\mathcal{R}} t \rightarrow_{\mathcal{R}} s_2$ and we want to make $\mathcal{R}$ be able to describe a theory of equality, we need to find some way to enforce **confluence**: because s_1 and s_2 ought to be equal if $\mathcal{R}$ is taken to be axiomatizing equalities, we need to ensure there is a term c such that $s_1 \rightarrow_{\mathcal{R}}^* c \leftarrow_{\mathcal{R}}^* s_2$. Ground Knuth-Bendix completion FiXme: cite is one method to do this, based on adding a new rewrite $s_1 \rightarrow_{\mathcal{R}} s_2$ to 'orient' each critical pair.

E-graphs take another approach: a nonconfluent critical pair $s_1 \leftarrow t \rightarrow s_2$ can only arise if there are rewrite rules that *overlap*, which we can put into two categories: if $f(\ldots) \rightarrow s_1$ and $f(\ldots) \rightarrow s_2$ can both apply to the same ground term, this will manifest in an E-graph as an E-node that is part of two E-classes, which will be merged on rebuilding. If $f(\ldots, g, \ldots) \rightarrow s_1$ and $g \rightarrow s_2$, then note that we will not actually represent the term $f(\ldots, g, \ldots)$ directly: rather, we will introduce a new name e_1 and write it as $f(\ldots, e_1, \ldots) \rightarrow s_1$. In this case, after taking into account the rewrite applied to g, the critical pair is resolved automatically, as if $g = s_2 = e_1$ then $f(\ldots, g, \ldots) = f(\ldots, s_2, \ldots) = f(\ldots, e_1, \ldots) = s_1$ will all follow from the presented equational theory.

So, the flattening that happens when building E-graphs can be seen as a process of introducing new names for every subterm, since every subterm is a *potential* critical pair. By introducing these names, we restrict the need to scan for critical pairs to the case of complete overlap: two equal terms in different E-classes.

Based on this, we can diagnose the problems of E-graphs on AC as the E-graph speculatively adding all 'AC-subterms' to the E-graph as any one of them could part of some critical pair, up to AC. For instance, if we have the terms $a+b+c+d$ and $c+e+b$, they have an overlap in the AC-subterm $b+c$, and if these terms were assigned to different E-classes, we would have the AC-critical pair $e_1 + e \leftarrow a+b+c+d+e \rightarrow a+d+e_2$.

So, while E-graphs are built around presenting a confluent ground rewriting system, the notion of EMT over a theory T suggests moving to the notion of confluent ground T-rewriting system in the natural way, in which case we need to pay special attention to critical pairs, which need to be resolved to maintain confluence.

FiXme: Some editing seams likely to appear in the next few subsections

4.1 EMTs: prior attempts

That *some* notion of EMT ought to exist has been stated multiple times before ([7], FiXme: blog post citations). The hope is to be able to use theory-specific knowledge to integrate certain identities directly into the procedures, rather than having to time-consumingly match, seed, and merge as directed by the identities. We will briefly go over the key intutition behind these proposals, and show that they fail to take into account the T-critical pair idea we outlined above.

4.1.1 Intuition 1: Canonizers

Zucker [7] notes that we have *canonizers* for many theories, which provide a way to solve the problem of ground equality modulo theories. A canonizer for a theory T is just some function on terms c such that, if $t_1 =_T t_2$, then

$c(t_1) = c(t_2)$: that is, it shifts the problem from equality modulo T to just ordinary term equality.

4.1.2 Intutition 2: Containers

The signature for an E-graph, Σ, is equivalent to a certain kind of *container*: one that has a number of 'shapes' (corresponding to the function symbols) and each shape has an ordered collection of slots (with length given by the arity of the shape). Thus, it is natural to consider generalizing this to containers of more general varieties. If one assigns to a single symbol one shape of every arity , with ordered slots, this gives the theory of AU: terms like $+()$, $+(a, b)$, $+(a, b, c)$ are all covered by this notion, and one doesn't need to arbitarily split a given list into binary uses of $+(-, -)$ and nilary uses of e. If the ordering constraint is dropped on the collections of slots, we get multiset containers, suitable for ACU, and moving to set containers gives something akin to ACIU – ACU with idempotence.

The second intuition also arises naturally out of concrete implementation work on E-graphs. Given a signature with an ACU binary operator f with unit e, and some other operators g, h, we can imagine building a parameterized data structure for term-fragments along the lines of the following pseudo-Haskell:

```
data TermF a
       = E
       | F a a
       | G a a a
       | H a
```

Looking at this structure and considering the ACU nature of f and e, it is only natural to change this to

```
data TermF a
       = Fs (Multiset a)
       | G a a a
       | H a
```

for an appropriate type constructor of multisets.

FiXme: Cite literature on polynomial functors/containers?

4.1.3 Canonizers and containers and the problem of flattening

The first intuition naturally connects to the second intuition if we consider the generalized idea of 'container' as representing some canonical form. Both approaches at first glance seem to take care of a lot of potential fiddling with identities in such theories. However, there is one issue: while we manage to

remove the some part of the theory with this approach, there is still one derived identity which we need to handle.

For instance, consider an AU pair of operators $+, 0$. The identities are

$$a + (b + c) = (a + b) + c \tag{4.1}$$
$$a + 0 = a \tag{4.2}$$
$$0 + a = a \tag{4.3}$$

Using a list representation, these become

$$[a, [b, c]] = [[a, b], c] \tag{4.4}$$
$$[a, []] = a \tag{4.5}$$
$$[[], a] = a \tag{4.6}$$

These identities are not entirely automatic just by switching to lists: we need the additional embedding and and flattening laws

$$a = [a] \tag{4.7}$$
$$[[a_1, \ldots, a_n], [b_1, \ldots, b_m], \ldots] = [a_1, \ldots, a_n, b_1, \ldots, b_m, \ldots] \tag{4.8}$$

Or in other words, every term can be embedded in a list of just that term, and a list of lists can be flattened by concatenating all of its elements.

Eq 4.7 and eq 4.8 together with the basic rules of lists are what is required to prove eq 4.4, eq 4.5, and eq 4.6.

FiXme: Cite literature on unbiased presentations? This is a monad law...

It is the need to handle embedding and flattening that gives rise to problems with an approach based on just canonizers or containers. We will then also see that embedding and flattening are the two rules that govern the formation of T-critical pairs.

A typical example of where these approaches break down is given by

$$a + b = b \tag{4.9}$$
$$a + a = c \tag{4.10}$$
$$c + b = d \tag{4.11}$$

taken modulo associativity of $+$.

The goal is to prove simply that $b = d$, which follows from the simple deduction

$$\begin{aligned} & b \\ & = a + b \\ & = a + (a + b) \\ & = (a + a) + b \\ & = c + b \\ & = d \end{aligned}$$

If we try the containers approach, we start by building up an E-graph with the following E-classes:

$e_1 = \{b, [a, b]\}, e_2 = \{c, [a, a]\}, e_3 = \{d, [c, b]\}$.

However, this is by itself, entirely inert. We need to use flattening to reason that, since $b = [a, b]$, that $[a, b] = [a, [a, b]] = [a, a, b]$. Then we need to use the converse of flattening to notice that $[a, a, b] = [[a, a], b] = [c, b] = d$.

For the canonizer approach, we can easily canonize the input equations: indeed, all of them have both sides in canonical form already. What is missing, then? Well, if we view the E-graph as a rewrite system, there is of course an A-critical pair between the rewrite $a+b \to e_1$ and $a+a \to e_2$. The critical pair is $a+a+b$, which can be rewritten as either $a + e_1$ or $e_2 + b$.

We conclude that canonizers and containers are not enough: they can rewrite terms that might appear in equations to a canonical form, but this is not enough to make the equations $s_i = t_i$ when flattened and oriented into the form $s_i \to^* e_j \leftarrow^* t_i$ to naturally form a T-confluent system rewrite.

There are some cases where canonizers or containers avoid this problem: for instance, consider the theory C. A critical pair mod C can only be of the form $e_1 \leftarrow a+b =_C b+a \to e_2$. If we always canonize (mod C) both sides of equations when we orient them (say, by sorting according to some stable order), then we will have a confluent system mod C. (This requires some care to maintain the ordering given that a and b might be E-class ids and subject to change via union-find and rebuilding).

FiXme: Backward ref to minimally expansive theories, to be renamed to weak acyclicity or whatever. Actually, conjecture: all minimally expansive/weakly acyclicity theories can be done with just a canonizer/container

4.2 EMTs and the word problem

4.2.1 The problem of matching modulo theories

Still, even with a canonizer or container, there is the problem that E-graph matching becomes matching modulo theories.

FiXme: Expand this. Also, how hard is it, in practice? For theories where a canonizer works/solves the word problem, is Zucker's bottom-up idea sufficient?

4.3 E-graphs modulo theorie

FiXme: prior contents: C, A, A revisited, AC, ACU

FiXme: Some more credit for prior work on EMTs: https://pavpanchekha.com/blog/lin-graphs.html, https://uwplse.org/2026/02/24/egglog-containers.html

4.3.1 Defining EMTs

Our general setup will be similar to E-graphs: FiXme: TODO: the formal definition of E-graphs should be lifted from the orphaned section to somewhere before this we will maintain a set of equivalence classes, which will contain E-nodes/term fragments over equivalence classes.

For a theory T, the T-part of the E-graph is the set of E-nodes that use a function symbol that appears in an identity in T.

For instance, if we have the E-class $e_1 = \{e_2+e_3, f(e2), e_3+e_4\}$, then the T-part of this, taking T to include only the $+$ symbol, is $e_1 = \{e_2 + e_3, e_3 + e_4\}$. An E-class that has no T-nodes still will appear in the form of $e_2 = \{\}$, recording that e_2 is T-*atomic*.

Our definition of EMTs will require a *decision procedure for the T-word problem*. That is, we need a procedure that, given the union of T and a set of equations, determines if another arbitrary equation follows from T and the equations.

Necessity of the word problem 1

We might imagine that we could restrict to the problem of determining if $e_i = e_j$ for equivalence classes e_i, e_j that actually appear in the E-graph. However, this restriction does not make anything easier: given an arbitrary equation $s = t$, we can insert s and t into the E-graph and then ask if the resulting equivalence classes are equal.

Necessity of the word problem 2

An E-graph in canonical form decides the word problem over uninterpreted function symbols. More concretely, an E-graphs describes a canonizer c such that $c(s) = e_i = c(t)$ iff $s = t$ follows from the input equations. It is also the case that, say, $c(f(s)) = f(e_i) = c(f(t))$ can be deduced by the E-graph even if $f(e_i)$ is not represented, but we can weaken this requirement for EMTs. We will consider just the very weak requirement that $s \to e_i \leftarrow t$ iff $s = t$ follows from the input equations and T, where $\to$ is a sequence of either rewrite rules from the EMT and applications of identities in T (and symmetrically for $\leftarrow$).

FiXme: Writing this makes me think we really ought to introduce $\vdash$ at some point for brevity. Also, check later if we have commented on the sufficiency of our requirements

Unfortunately, this rules out EM(A) and EM(AU), given that the relevant word problems are undecidable.

Also, we need to extend matching to the more general matching modulo T, given that the T-part now won't necessarily represent all terms up to T in a 'direct' way. (The hope of being able to represent E-graphs under T in less space than standard E-graphs is, of course, the motivation for EMTs to begin with.)

FiXme: Once I have re-homed the orphaned formalization pieces, finish this with a formalization of EMTs

A sketch of the formalization: The solver maintains its own view of the T-part of the EMT. When nodes are merged via congruence-closure canonicalization/an explicit merging, that generates an event asking the solver to merge the T-part of the relevant equivalence classes. The solver can, in turn, send a discovered equality between two equivalence classes back to the E-graph section.

When nodes are seeded, if they are headed by a function symbol mention in T, then the solver gets a relevant seed event, of course. When given an input term, the solver acts as a canonizer to rewrite the term into a canonical form, which can be tested for equality or seeded. The lack of critical pairs in the rewriting step is what makes a solver good. I need to go over nelson-oppen theory combination here probably

Chapter 5

E-graphs modulo AC

FiXme: Make sure we have cited 'congruence closure mod AC' by Bachmair and the Cochon paper as very relevant prior work. Tiwari also

5.1 Solving the word problem modulo AC

A single AC operator forms a *commutative semigroup*. We have a remarkably tight bound on the asymptotic complexity of solving the word problem over commutative semigroups: Mayr and Meyer [2] prove that it is EXPSPACE-complete, that is to say, it requires $O(2^n)$ space to compute. This result also provides an embeddding of the word problem into the *polynomial ideal* word problem, a problem that is solvable via finding *Gröbner bases*. Finding Gröbner bases is central problem in computer algebra which is also EXPSPACE-complete, and so it is both asymptotically optimal for solving AC and, we hope, the best-optimized approach *in practice* for solving the AC word problem. Moreover, the same algorithm can be used for solving various variants of AC, such as ACU, commutative rings, and polynomials over commutative rings. Therefore, we focus on Gröbner basis methods as a key method that provides a word-problem solver for a wide range of theories of interest.

5.1.1 Polynomial ideals and the AC embedding

Take a finite set $X = \{e_1, \ldots, e_n\}$. Then two terms over X built from an AC operator f are AC-equal if they have the same number of each element of X in any order; that is to say, we take them equal as *multisets*.

Instead of writing a term as $f(e_1, f(e_1, f(e_2))$, we will instead write it as $e_1^2 e_2$, to connect better with the standard notation for polynomials.

The set of **polynomials** $R[X]$ over X and the ring of coefficients R and is the set of R-linear combinations of AC terms over X: for instance, if $r_1, r_2 \in R$, then $r_1 e_1^2 e_2 + r_2 e_3$ is a polynomial in $R[X]$.

Given a finite set of equations E between X-terms of the form $\alpha_i = \beta_i$, we can construct the set $I(E) = \{\sum_i g_i(\alpha_i - \beta_i) \mid g_i \in R[X], E_i = (\alpha_i = \beta_i)\}$.

Then, E implies that two X-terms s and t are equal iff $s - t \in I(E)$.

FiXme: TODO: proof

$I(E)$ is an ideal in the set of polynomials

5.1.2 Gröbner bases for ideals

5.1.3 Computing Gröbner bases via Buchberger's algorithm

5.2 Matching modulo AC

Chapter 6

Relational queries

FiXme: Section commented out for now, as it this section is very WIP and has taken a back-seat to other issues

Chapter 7

An implementation

Chapter 8

Prior work

Chapter 9

Conclusion and future work

Chapter 10

Plan

10.1 Theoretical developments and literature review

Mostly complete

10.1.1 Implementation

Implement the infrastructure of the E-graph library: joins, join plans, the E-graph structure, and a saturation loop

Target date: Monday, week 15 (6th of April)

Implement the modulo-theory algorithm into the library

Target date: Monday, week 16 (13th of April)

10.1.2 Benchmarking

10.1.3 Collect a benchmark suite of problems

I will need to decide if time-to-saturation is a good metric, as many existing cases don't run to saturation, and in many cases this is provably impossible. It is likely we will need to use the somewhat less precise metric of time to prove a particular desired 'target' equation, which is more dependent on ordering of phases and so forth.

Target date: Wednesday, week 17 (22nd of April)

10.1.4 Run benchmarks

We want to try both various join plans and with/without the modulo-theories aspect. It will be important to see to what extent the various join plans make a difference, and also to test out the effectiveness of Zucker's incomplete matcher.

Target date: Wednesday, week 18 (29th of April)

10.1.5 Finishing the report and giving the presentation

Remainder of time in May

10.2 Limitations of the plan

10.2.1 Detailed microbenchmarking

I am hoping that algorithmic improvements will be sufficient to cause noticeable differences in the benchmark scores without worrying too much about the details of the micro-optimizations that might be applicable.

10.2.2 Flaws in various prior versions of the proposal and plan

I have realized that a number of technical claims made previously are flawed for a variety of reasons.

Note: this section is for historical interest, and now some better versions of this are in the thesis proper.

Asymptotics of A/C

In general, I have referred to 'exponential blowup' in representing all the terms implied by the AC identities in an E-graph.

- For C, any term +(a, b) implies the term +(b, a) in the same E-class. This means that for n + nodes, there are $2n$ nodes taking into account C. This is hardly a blowup.

- For A, we can think of a term up to associativity as a string of its non-+ leaf nodes. The number of binary trees that yield a string of length $n+1$ is given by the n-th Catalan number, which by itself is exponential in n. However, due to the sharing of subterms, things are hardly as bad as this. In general, each of the $O(n^2)$ substrings of the input string will be given a single E-class, each containing $O(n)$ E-nodes for the various ways to divide that substring into two parts. If we allow cyclic terms modulo A – for instance, we represent the equation $a = a + b$ – then the number of

reassociations becomes infinite. Thus, for the non-cyclic case, we have a size proportional to $O(n^3)$, and in the cyclic case, an infinite size.

- For AC, a term up to AC can be thought of as a multiset of non-+ leaf nodes. The worst case is when all the leaves are distinct, in which case there are $2^n - 1$ nonempty-sub-multisets of the input term, or 2^n if we are working in ACU. This is the case where the size of a naïve representation truly is exponential. Worse, we don't just have $O(2^n)$ E-classes, but several E-nodes per class to represent every multiset decomposition $u = v + w$ applicable to each E-class, which gives an average size of 2^n to the E-classes and $O(2^n 2^n) = O(2^{2n})$ to the E-graph.

  Curiously, the factor $n!$ that one might expect might be relevant to the combinatorics of either C or AC never seems to come up, except indirectly via the identity $2^n = \sum_k \frac{n!}{k!(n-k)!}$.

Zucker's proposal

Zucker's proposal used the idea of 'bottom-up matching' together with a canonizer as an implementation of EMT. However, this is not complete for A or AC. A major initial focus of mine was to formalize matching in a way that would allow a faster method than the naïve bottom-up method, as well as understanding Zucker's notion of a 'theory factor'.

In discussion with Zucker, I have now determined that the 'theory factor' notion is rather suspect and that his algorithm is incomplete. Therefore, the problem of matching and my ideas around efficient joins are for now taking a back stage, although I still intend to write up some relevant results in this field.

EMT(A)

For a while, I was convinced that EMT(A) was decidable based on an incorrect connection to the decidable unification-mod-A problem. However, I now believe this is not decidable. I intend to see to what extent this lack of decidability can be mitigated.

Bibliography

[1] James Koppel. *Version Space Algebras are Acyclic Tree Automata*. July 27, 2021. DOI: 10.48550/arXiv.2107.12568. arXiv: 2107.12568[cs]. URL: http://arxiv.org/abs/2107.12568 (visited on 04/14/2025).

[2] Ernst W Mayr and Albert R Meyer. "The complexity of the word problems for commutative semigroups and polynomial ideals". In: *Advances in Mathematics* 46.3 (Dec. 1, 1982), pp. 305–329. ISSN: 0001-8708. DOI: 10.1016/0001-8708(82)90048-2. URL: https://www.sciencedirect.com/science/article/pii/0001870882900482 (visited on 03/11/2026).

[3] Natarajan Shankar. "Little Engines of Proof". In: *FME 2002:Formal Methods—Getting IT Right*. Ed. by Lars-Henrik Eriksson and Peter Alexander Lindsay. Red. by Gerhard Goos, Juris Hartmanis, and Jan Van Leeuwen. Vol. 2391. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 1–20. ISBN: 978-3-540-43928-8 978-3-540-45614-8. DOI: 10.1007/3-540-45614-7_1. URL: http://link.springer.com/10.1007/3-540-45614-7_1 (visited on 03/17/2026).

[4] Dan Suciu, Yisu Remy Wang, and Yihong Zhang. *Semantic foundations of equality saturation*. Jan. 5, 2025. DOI: 10.48550/arXiv.2501.02413. arXiv: 2501.02413[cs]. URL: http://arxiv.org/abs/2501.02413 (visited on 02/04/2026).

[5] Yisu Remy Wang et al. *E-Graphs, VSAs, and Tree Automata: a Rosetta Stone*. URL: https://remy.wang/reports/dfta.pdf.

[6] Max Willsey et al. "egg: Fast and extensible equality saturation". In: *Proceedings of the ACM on Programming Languages* 5 (POPL Jan. 4, 2021), pp. 1–29. ISSN: 2475-1421. DOI: 10.1145/3434304. URL: https://dl.acm.org/doi/10.1145/3434304 (visited on 02/08/2026).

[7] Philip Zucker. *Omelets Need Onions: E-graphs Modulo Theories via Bottom-up E-matching*. Apr. 19, 2025. DOI: 10.48550/arXiv.2504.14340. arXiv: 2504.14340[cs]. URL: http://arxiv.org/abs/2504.14340 (visited on 01/20/2026).